

If a $\mathbf{P}$ exists such that $\tau([\mathbf{P}, \mathbf{X}_i]) = \mathbf{Y}_i$ holds for both $i = 1, 2$, then we have

$$\begin{aligned}\text{Att}(\mathbf{x}_0, [\mathbf{P}, \mathbf{X}_1]) &= \lambda(\mathbf{X}_1, \mathbf{x}_0, [\mathbf{P}, \mathbf{X}_1])\text{Att}(\mathbf{x}_0, \mathbf{X}_1) + \lambda(\mathbf{P}, \mathbf{x}_0, [\mathbf{P}, \mathbf{X}_1])\text{Att}(\mathbf{x}_0, \mathbf{P}) \\ \text{Att}(\mathbf{x}_0, [\mathbf{P}, \mathbf{X}_2]) &= \lambda(\mathbf{X}_2, \mathbf{x}_0, [\mathbf{P}, \mathbf{X}_2])\text{Att}(\mathbf{x}_0, \mathbf{X}_2) + \lambda(\mathbf{P}, \mathbf{x}_0, [\mathbf{P}, \mathbf{X}_2])\text{Att}(\mathbf{x}_0, \mathbf{P})\end{aligned}\quad (8)$$

where $\lambda(\cdot, \cdot, \cdot)$ is a positive scalar. We also have

$$\begin{aligned}\text{Att}(\mathbf{x}_0, [\mathbf{P}, \mathbf{X}_1]) + \mathbf{x}_0 &\in \text{MLP}^{-1}(\mathbf{y}_{10}) \\ \text{Att}(\mathbf{x}_0, [\mathbf{P}, \mathbf{X}_2]) + \mathbf{x}_0 &\in \text{MLP}^{-1}(\mathbf{y}_{20})\end{aligned}$$

as $\text{MLP}(\text{Att}(\mathbf{x}_0, [\mathbf{P}, \mathbf{X}_i]) + \mathbf{x}_0) = \mathbf{y}_{i0}, i = 1, 2$.

Therefore, $\text{Att}(\mathbf{x}_0, \mathbf{P})$ must be in both $\text{Cone}(\mathbf{a} - \mathbf{x}_0, -\text{Att}(\mathbf{x}_0, \mathbf{X}_1))$ and $\text{Cone}(\mathbf{b} - \mathbf{x}_0, -\text{Att}(\mathbf{x}_0, \mathbf{X}_2))$, where $\mathbf{a} \in \{\mathbf{y}_{10}, (\mathbf{W}_2\mathbf{W}_1 + \mathbf{I})^{-1}\mathbf{y}_{10}\}$ and $\mathbf{b} \in \{\mathbf{y}_{20}, (\mathbf{W}_2\mathbf{W}_1 + \mathbf{I})^{-1}\mathbf{y}_{20}\}$, which contradicts the existence of $\mathbf{P}$ as $\text{Cone}(-\text{Att}(\mathbf{x}_0, \mathbf{X}_1), \mathbf{a} - \mathbf{x}_0) \cap \text{Cone}(-\text{Att}(\mathbf{x}_0, \mathbf{X}_2), \mathbf{b} - \mathbf{x}_0) = \emptyset$. Therefore, in this example, even though we allow an arbitrary number of trainable parameters in prompt $\mathbf{P}$, we cannot find one to exactly memorize the training set with only two training examples.

This theorem reveals an important difference between prompt tuning and adjusting the model weights directly. For any training dataset S with two training examples $\{(\mathbf{X}_1, \mathbf{Y}_1), (\mathbf{X}_2, \mathbf{Y}_2)\}$, so long as $\text{Att}(\mathbf{X}_1, \mathbf{X}_1) + \mathbf{X}_1$ and $\text{Att}(\mathbf{X}_2, \mathbf{X}_2) + \mathbf{X}_2$ are distinct, MLP can easily map the post-attention features to expected output tokens with finite number of hidden neurons. As a result, tuning the MLP parameters for this pretrained transformers can memorize any dataset in the form of Assumption 1. However, prompt tuning cannot achieve this even if the number of tunable tokens $\rightarrow$ infinity, thereby limiting the expressiveness of prompt tuning when compared to model fine-tuning.

5.2 Comparison with a More General Dataset

In Section 5.1, we constructed sequence-to-sequence datasets that cannot be learned by a given transformer layer with prompt tuning, by utilizing the shared token between different training examples. In this section, we compare the expressive power of prompt tuning and fine-tuning under a more general dataset construction where the former requirement can be relaxed.

Since the primary essence of prompt tuning is to perform parameter-efficient tuning, wherein we seek to adapt a pretrained large model to a new task with fewer tunable parameters, we compare prompt tuning with another parameter-efficient version of model-tuning: LoRA [Hu et al., 2021]. Succinctly, we compare the required number of parameters to memorize a given dataset. Again, consider a sequence-to-sequence dataset $S = \{(\mathbf{X}_1, \mathbf{Y}_1), (\mathbf{X}_2, \mathbf{Y}_2), \dots, (\mathbf{X}_n, \mathbf{Y}_n)\}$, where $\mathbf{X}_i = [\mathbf{x}_{i1}, \mathbf{x}_{i2}, \dots, \mathbf{x}_{im}]$ and $\mathbf{Y}_i = [\mathbf{y}_{i1}, \mathbf{y}_{i2}, \dots, \mathbf{y}_{im}]$. We again discuss the memorization of the last output token for simplicity and results can be directly extended.

We first give the required number of parameters of LoRA to memorize dataset S .

Lemma 5 (LoRA). *For a standard single-layer transformer τ defined in Definition 2 with $r \geq n$ MLP hidden neurons, for any sequence-to-sequence dataset S satisfying Assumptions 1, we can apply a low-rank update to MLP weights with $O(nd)$ parameters to memorize $\tau(\mathbf{X}_i)_{:,m} = \mathbf{y}_{im}$.*

This lemma is derived based on the memorization capabilities of 1-hidden layer MLPs [Yun et al., 2019b]. As the post-attention values for different training inputs are different from Assumption 1, we can construct a low rank update with $O(nd)$ parameters on the MLP layer to memorize S . We defer the complete proof to Appendix C.7.

For prompt tuning, we derive a result in the next theorem which shows that it requires $\Omega(nd)$ tunable parameters to memorize some constructed dataset S with n examples.

Theorem 3 (Lower bound on Tunable Prompt Parameters). *For any single layer transformer τ defined in Definition 2, there exists a sequence-to-sequence dataset $\{(\mathbf{X}_1 = [\mathbf{x}_{10}, \mathbf{x}_1], [\mathbf{y}_{10}, \mathbf{y}_{11}]), (\mathbf{X}_2 = [\mathbf{x}_{20}, \mathbf{x}_2], [\mathbf{y}_{20}, \mathbf{y}_{21}]), \dots, (\mathbf{X}_n = [\mathbf{x}_{n0}, \mathbf{x}_n], [\mathbf{y}_{n0}, \mathbf{y}_{n1}])\}$ that satisfies Assumption 1 with $n < d$ training examples such that we need at least n prompt tokens in $\mathbf{P}$ to memorize the training set, ie, for $\tau([\mathbf{P}, \mathbf{X}_i])_{:,-1} = \mathbf{y}_{i1}$ to hold for all $i = 1, 2, \dots, n$.*

This dataset can be constructed by including n examples that require n linearly independent prompts tokens. The complete proof is deferred to Appendix C.8.

Note that in Theorem 3, we provide a key lower bound on the required number of prompt tokens for exact memorization and this can very well more than nd . This partially (but not necessarily) explains the worse empirical performance of prompt tuning against LoRA under a comparable number of trainable parameters.

6 Extension to Multi-Layer Setting

In this section, we extend our analysis to multi-layer setting and provide a sufficient condition under which the expressiveness of prompt tuning is restricted. An immediate consequence of our result is an interesting connection to the spectral norm of soft prompts surfaces. This result provides us a partial understanding of the phenomenon that soft prompt $\mathbf{P}$ vectors typically exhibit larger norms compared to the actual input $\mathbf{X}$, after the tuning.

With some further notation adjustments, we denote an H layer pretrained transformer network as $g(\in \mathcal{T}) = \tau^1 \circ \tau^2 \circ \dots \circ \tau^H$, the input set as $\mathcal{X}^1$, and the set of possible prompts as $\mathcal{P}^1$. We assume that the following compactness condition is satisfied:

$$\begin{aligned} \|\mathbf{P}^l, \mathbf{X}^l\|_2 &\leq D^l \\ \text{s.t. } [\mathbf{P}^{l+1}, \mathbf{X}^{l+1}] &= \tau^l([\mathbf{P}^l, \mathbf{X}^l]), \forall l = 1, \dots, H. \end{aligned} \quad (9)$$

Here $[\mathbf{P}^1, \mathbf{X}^1]$ is the input to the first layer τ^1 with $\mathbf{P}^1 \in \mathcal{P}^1$, $\mathbf{X}^1 \in \mathcal{X}^1$ and $\|\cdot\|_2$ is the spectral norm. Similarly, $[\mathcal{P}^{H+1}, \mathcal{X}^{H+1}]$ denotes the output set.

We start by providing an upper bound to the Lipschitz constant of attention, pertaining to eq 9. This derivation is different from the works of [Dasoulas et al., 2021, Vuckovic et al., 2020] and thus can be of independent interest.

Lemma 6. *Under the compactness condition, the Lipschitz constant of the i -th attention head in the l -th transformer layer, denoted for simplicity as $\text{Att}^{i,l}$, admits the following bound w.r.t the entire input sequence of length m :*

$$\text{Lip}(\text{Att}^{i,l}(\cdot, \cdot)) \leq (1 + 8\sqrt{m}(D^l)^2 \|(\mathbf{W}_k^{i,l})^T \mathbf{W}_q^{i,l}\|_2) \|\mathbf{W}_v^{i,l}\|_2, \quad (10)$$

and the Lipschitz constant of the entire attention block in layer l , denoted as Att^l , admits the bound:

$$\text{Lip}(\text{Att}^l(\cdot, \cdot)) \leq \sqrt{\sum_{i=1}^h (\|\mathbf{W}_o^{i,l}\|_2 \times \text{Lip}(\text{Att}^{i,l}))^2}. \quad (11)$$

It is noteworthy that this upper bound is dependent on D^l , the spectral norm of the input prepended with the prompt. In conjunction with the following theorem, we obtain a result on limited expressivity of prompt tuning by showing that the transformer becomes invertible, in consideration to functions from $\mathcal{P}^1 \times \mathcal{X}^1 \rightarrow \mathcal{P}^{H+1} \times \mathcal{X}^{H+1}$ (an extension to functions of the form $\mathcal{X}^1 \rightarrow \mathcal{X}^{H+1}$ is provided in Appendix Section C.11).

Theorem 4. *A transformer $g \in \mathcal{T}$ is invertible, ie, $g^{-1}(\mathbf{Y}) = \{\mathbf{X} : g(\mathbf{X}) = \mathbf{Y}\}$ is a singleton set $\forall \mathbf{Y}$ in range of g , if:*

1. *The Lipschitz constant of the attention block in each layer τ^l is strictly less than 1*
2. *The Lipschitz constant of the 2-layer ReLU block in each layer τ^l , which is bounded by $\|\mathbf{W}_2^l\|_2 \times \|\mathbf{W}_1^l\|_2$, is strictly less than 1.*

Proof of Theorem 4 can be found in Appendix C.9. Combining Lemma 6 and Theorem 4, we observe that the invertibility is guaranteed if the upper bound for the Lipschitz constant of the attention, eq 11, and the MLP layer, is strictly less than 1. In this case, we can then construct arbitrarily many datasets where two different inputs share the same output, and prompt tuning cannot learn (more subtly: memorize) these datasets with a restricted prompt norm.

7 Experiments

7.1 Experimental Settings

In Section 7.2, we use a standard single-layer single-head transformer from Definition 2, to justify the infinite prompt-length limitation. In Section 7.3, we justify the increasing prompt norm on the pretrained LLaMA 7B model [Touvron et al., 2023]. For prompt tuning and LoRA, we use the Huggingface Peft library [Mangrulkar et al., 2022]. On the dataset front, we utilize the RTE subtask of SuperGlue dataset [Wang et al., 2019] and WMT14 En-Fr translation [Bojar et al., 2014]. More details and hyperparameter settings can be found in Appendix A.

7.2 Limited Expressivity of Infinite Length Prompt

We first construct the dataset following the proof of Theorem 2 and then show that prompt tuning cannot memorize this simple dataset $\{(\mathbf{X}_1 = [\mathbf{x}_1, \mathbf{x}_0], \mathbf{Y}_1 = [\mathbf{y}_{11}, \mathbf{y}_{10}]), (\mathbf{X}_2 = [\mathbf{x}_2, \mathbf{x}_0], \mathbf{Y}_2 = [\mathbf{y}_{21}, \mathbf{y}_{20}])\}$ even with very large prompt lengths.

We set the token dimension $d = 10$. We follow the default pytorch weight initialization and then normalize $\mathbf{W}_1, \mathbf{W}_2$ such that $\|\mathbf{W}_2\|_2 \times \|\mathbf{W}_1\|_2 < 1$, following Assumption 2. We randomly sample $\mathbf{x}_0, \mathbf{y}_{10}, \mathbf{y}_{20}$ in a uniform distribution in $[0, 1]^d$ and construct the corresponding vectors: $\mathbf{x}_1$ and $\mathbf{x}_2$ following Theorem 2. To compute $\text{MLP}^{-1}(\mathbf{y})$, we follow [Kim et al., 2021] Section 4.1 with 5000 iterations at convergence. We solve $\text{Att}(\mathbf{x}_0, [\mathbf{x}_0, \mathbf{x}_1]) \parallel \mathbf{c}$ in Lemma 7 with gradient descent terminating at $\angle(\text{Att}(\mathbf{x}_0, [\mathbf{x}_0, \mathbf{x}_1]), \mathbf{c}) < 0.0001$. We repeat this setup to obtain 3 different datasets for distinct $\mathbf{x}_0, \mathbf{y}_{10}, \mathbf{y}_{20}$ and denote these with $S_i, i = 1, 2, 3$.

We perform prompt tuning, MLP fine-tuning and MLP LoRA training on the constructed datasets for 5 runs and report the mean and standard deviation of per-element Mean Squared Error (MSE) loss $\mathbf{y}_{10}, \mathbf{y}_{20}$ at convergence. We show the comparison between prompt-tuning and MLP fine-tuning in Figure 1. As we can observe from the figure, increasing the number of soft prompt tokens post a certain threshold that does not exhibit any reduction in MSE. On the contrary, fine-tuning on the MLP layer tend to easily memorize the training set by reducing the training loss to almost zero (all the three curves for fine-tuning overlap and thus not differentiated). Note that we plot the standard deviation, however it is negligible in the range. Similar to fine-tuning on the MLP layer, LoRA with width 2 on the MLP layer also achieves near-zero training loss which is less than 10^{-10} on the constructed dataset. We don't plot the comparison on Figure 1 as all the six curves are overlapped). This result validates our Theorem 3 that LoRA can memorize a dataset with n examples with trainable parameters $O(n)$ while prompt-tuning may require more.

7.3 Increasing Prompt Spectral Norm during Tuning

As discussed in Section 6, a major constraint on the expressive power of prompt tuning is the spectral norm of soft prompts. In Figure 2, we plot the curve for spectral norm of soft prompt as training progresses and the loss reduces on RTE dataset. The curve for WMT14 En-Fr dataset can be found in Appendix B. This trend clearly highlights that in order to counter the limit on the capacity, the spectral norm consistently increases till the training loss saturates.

8 Conclusions

In this work, we embark on exploring the capabilities of prompt tuning in the continuous regime, contrasting it with fine-tuning, as an initial endeavor towards a theoretical comprehension. We prove by construction that prompt tuning admits universal approximation within the space of Lipschitz functions. Additionally, we identified inherent limitations of prompt tuning on single-layer transformers by constructing theoretically difficult datasets for prompt tuning. These limitations are then extended to multi-layer setting under a specific prompt-norm restriction.

From the analysis in Theorem 2 and 3, we note that the limitation of prompt-tuning primarily arises from the correlation across different inputs. Broadly describing, prompt-tuning implements transformation on different inputs via “additional attention values”, which is more restrictive as compared to the transformations from MLP layers on input tokens. An interesting potential direction to improve prompt-tuning is: “designing a mechanism to leverage prompting in order to generate